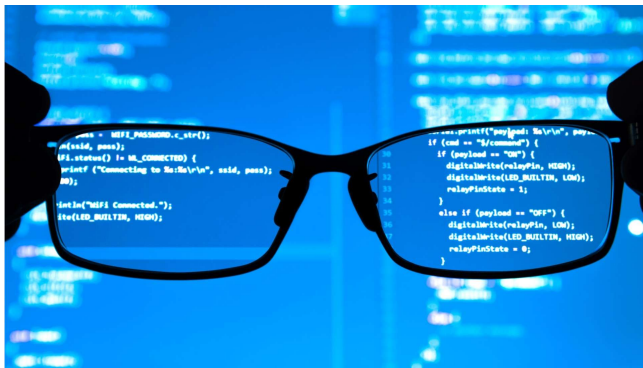




“EXPLORANDO LA IMAGEN DIGITAL CON PYTHON”

PRACTICA 1

OBJETIVO

Aplicar conceptos fundamentales del análisis de imágenes digitales mediante el desarrollo colaborativo de scripts en Python que permitan leer, transformar y analizar imágenes, relacionando los modelos de color con el procesamiento básico de imágenes

Dra. María Elena Cruz Meza
Análisis de imágenes

Práctica de Laboratorio: “Explorando la Imagen Digital con Python”

Objetivo General

Aplicar conceptos fundamentales del análisis de imágenes digitales mediante el desarrollo colaborativo de scripts en Python que permitan leer, transformar y analizar imágenes, relacionando los modelos de color con el procesamiento básico de imágenes.

Competencias Específicas

- Comprender el funcionamiento del sistema de visión humano y su relación con los sistemas artificiales.
- Identificar y aplicar modelos de color en imágenes digitales.
- Manipular imágenes digitales en Python: lectura, escalamiento, separación de componentes y binarización.
- Colaborar en equipos para resolver problemas técnicos y documentar soluciones.

Temas Abordados (Unidad I)

- 1.1. El sistema de visión humano
- 1.2. Modelos de color: RGB, CMY, YIQ, HSI, HSV
- 1.3. Obtención de imágenes digitales y el pixel
- 1.4. Lectura de imágenes mediante Python
- 1.5. Separación de componentes RGB
- 1.6. Escalamiento a grises
- 1.7. Binarización de imágenes

Estructura de la Práctica

Se trabajarán tres sesiones para evaluar la unidad 1.

Sesión 1 (90 min): Fundamentos y lectura de imágenes

- i. Breve repaso del sistema de visión humano y modelos de color.
- ii. Introducción a la biblioteca Pillow y OpenCV en Python.

La **biblioteca Pillow** en Python es un proyecto de código abierto que ofrece herramientas para manipular y procesar imágenes, siendo una **bifurcación (fork)** activa y mejorada de la antigua y descontinuada Python Image Library (PIL). Con Pillow, puedes abrir, guardar y realizar diversas operaciones sobre diferentes formatos de imagen (como JPEG, PNG, GIF), incluyendo redimensionar, recortar, rotar, aplicar filtros, añadir texto y convertir formatos, facilitando tareas de procesamiento de imágenes en Python para aplicaciones de nivel básico a intermedio.

Funciones principales de Pillow:

- **Compatibilidad con formatos:** Soporta múltiples formatos de archivo de imagen como JPEG, PNG, GIF, TIFF y BMP.
- **Descargar biblioteca:** <https://recursospython.com/guias-y-manuales/instalar-pil-pillow-efectos/>

- Documentación: <https://translate.google.com/translate?u=https://pillow.readthedocs.io/&hl=es&sl=en&tl=es&client=srp>

La biblioteca **OpenCV** en Python es una [Open Source Computer Vision Library](#) para el procesamiento de imágenes y visión artificial, que permite identificar objetos, rostros y escritura, así como realizar operaciones como filtrado y detección de bordes en videos e imágenes. Se utiliza en robótica, realidad aumentada y vehículos autónomos, y se instala con el comando `pip install opencv-python`.

¿Qué es OpenCV en Python?

- **Visión Artificial y Procesamiento de Imágenes:** OpenCV es una biblioteca de código abierto para la visión por computadora y el aprendizaje automático.
 - **Aplicaciones:** Permite crear aplicaciones para reconocimiento facial, detección de objetos, análisis de video en tiempo real, y realidad aumentada.
 - **Multilinguaje:** Aunque se usa mucho en Python, su implementación original en C++ lo hace compatible con otros lenguajes como C++, Java y MATLAB.
- Funcionalidades clave**
- **Procesamiento de Imágenes:** Permite realizar tareas como filtrado, detección de bordes y transformación de imágenes.
 - **Reconocimiento y Detección:** Se utiliza para identificar objetos, rostros, escritura a mano y para la extracción de características.
 - **Manejo de Videos:** Ofrece herramientas para el análisis de videos en tiempo real.

Cómo instalar OpenCV en Python

1. **Instalar Pip:** Asegúrate de tener el gestor de paquetes pip para Python instalado, ya que es esencial para descargar librerías.
2. **Instalar OpenCV:** Abre tu terminal o símbolo del sistema y ejecuta el siguiente comando para instalar la biblioteca:

```
bash
pip install opencv-python
```

Requisitos previos y uso con otras librerías

- **Numpy:** Es una biblioteca fundamental, ya que OpenCV utiliza matrices Numpy para sus operaciones, lo que permite una mayor eficiencia.
- **Entornos Virtuales:** Se recomienda usar entornos virtuales para aislar proyectos y evitar conflictos entre diferentes versiones de paquetes de Python.

- iii. Lectura de imágenes y visualización de componentes RGB.
- iv. Discusión sobre el pixel y la representación digital.

Estrategias activas:

- Estudio de caso
- Codificación en pares

Aprendizaje basado en problemas

Producto esperado:

- Script que lea una imagen y separe sus componentes RGB.
- Bitácora de trabajo colaborativo.

Sesión 2 (90 min): Transformaciones básicas

- Conversión de imágenes RGB a escala de grises.
- Aplicación de binarización con umbral fijo y automático.
- Visualización de histogramas de intensidad.

Estrategias activas

- Resolución de problemas en equipo
- Aprendizaje entre pares
- Mini-retos

Producto esperado:

- Script que escale a grises y binarice imágenes.
- Comparación de resultados con distintos modelos de color.

Sesión 3 (90 min): Presentación y reflexión

- Presentación de resultados por equipo.
- Retroalimentación entre pares.
- Reflexión sobre el proceso de trabajo colaborativo.

Estrategias activas:

- Presentación colaborativa
- Evaluación entre pares
- Reflexión guiada

Producto esperado:

- Informe técnico del proyecto.
- Repositorio en GitHub con código y documentación.

Aprendizaje Autónomo (3 horas)

- Lectura de documentación de Pillow y OpenCV.
- Mejora del código y documentación.
- Preparación de la presentación final.
- Reflexión escrita sobre el trabajo en equipo y los modelos de color.

Rúbrica de Evaluación

Criterio	Excelente (10)	Bueno (8)	Regular (6)	Deficiente (4)
Funcionalidad del código	Código completo, sin errores	Código funcional con mínimos errores	Código incompleto o con errores	Código no funcional
Aplicación de modelos de color	Uso correcto y comparación entre modelos	Uso adecuado de al menos un modelo	Uso limitado o incorrecto	Sin aplicación
Transformaciones de imagen	Escalamiento y binarización bien implementados	Transformaciones básicas funcionales	Transformaciones incompletas	Sin transformaciones
Trabajo colaborativo	Evidencia clara de colaboración	Colaboración parcial	Colaboración limitada	Sin colaboración
Presentación y documentación	Clara, profesional y completa	Clara pero con detalles faltantes	Poco clara o incompleta	Ausente o deficiente
Reflexión y mejora autónoma	Reflexión profunda y mejoras evidentes	Reflexión adecuada	Reflexión superficial	Sin reflexión

Anexos

A. Código Base en Python

Indicaciones

1. **Coloca una imagen en el mismo directorio** del script y cambia el nombre en ruta_imagen.
2. **Ejecuta el script paso a paso**, observando los resultados de cada transformación.
3. **Modifica el umbral de binarización** y analiza cómo cambia la imagen.
4. **Agrega funciones nuevas** para aplicar otros modelos de color (por ejemplo, HSV con OpenCV).
5. **Documenta tu código** y sube tu proyecto a GitHub.

A continuación se presenta el código base que deben utilizar como punto de partida para la práctica. Este script permite leer una imagen, separar sus componentes RGB, convertirla a escala de grises y binarizarla.

```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
import cv2
```

```
def cargar_imagen(ruta):
    imagen = Image.open(ruta)
    plt.imshow(imagen)
    plt.title("Imagen original")
    plt.axis("off")
    plt.show()
    return imagen
```

```
def separar_rgb(imagen):
    r, g, b = imagen.split()
```

```

plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.imshow(r, cmap='Reds')
plt.title("Componente R")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(g, cmap='Greens')
plt.title("Componente G")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(b, cmap='Blues')
plt.title("Componente B")
plt.axis("off")
plt.show()

def convertir_a_grises(imagen):
    imagen_cv = cv2.cvtColor(np.array(imagen), cv2.COLOR_RGB2GRAY)
    plt.imshow(imagen_cv, cmap='gray')
    plt.title("Imagen en escala de grises")
    plt.axis("off")
    plt.show()
    return imagen_cv

def binarizar_imagen(imagen_gris, umbral=128):
    _, binaria = cv2.threshold(imagen_gris, umbral, 255, cv2.THRESH_BINARY)
    plt.imshow(binaria, cmap='gray')
    plt.title(f"Imagen binarizada (umbral = {umbral})")
    plt.axis("off")
    plt.show()

if __name__ == "__main__":
    ruta_imagen = "imagen_ejemplo.jpg"
    imagen = cargar_imagen(ruta_imagen)
    separar_rgb(imagen)
    imagen_gris = convertir_a_grises(imagen)
    binarizar_imagen(imagen_gris, umbral=128)

```

B. Guía de Instalación de Python y Librerías

1. Instala Python desde <https://www.python.org/downloads/>
2. Abre una terminal o consola de comandos.
3. Instala las librerías necesarias ejecutando:
`pip install pillow matplotlib opencv-python`
4. Verifica que puedes ejecutar el script sin errores.